Universidad de los Andes

Tecnología e Infraestructura de Cómputo

Caso 3

Kaila Azur Zenkner - 202616968

Maria Jose Caro Romero - 202420310

Diseño

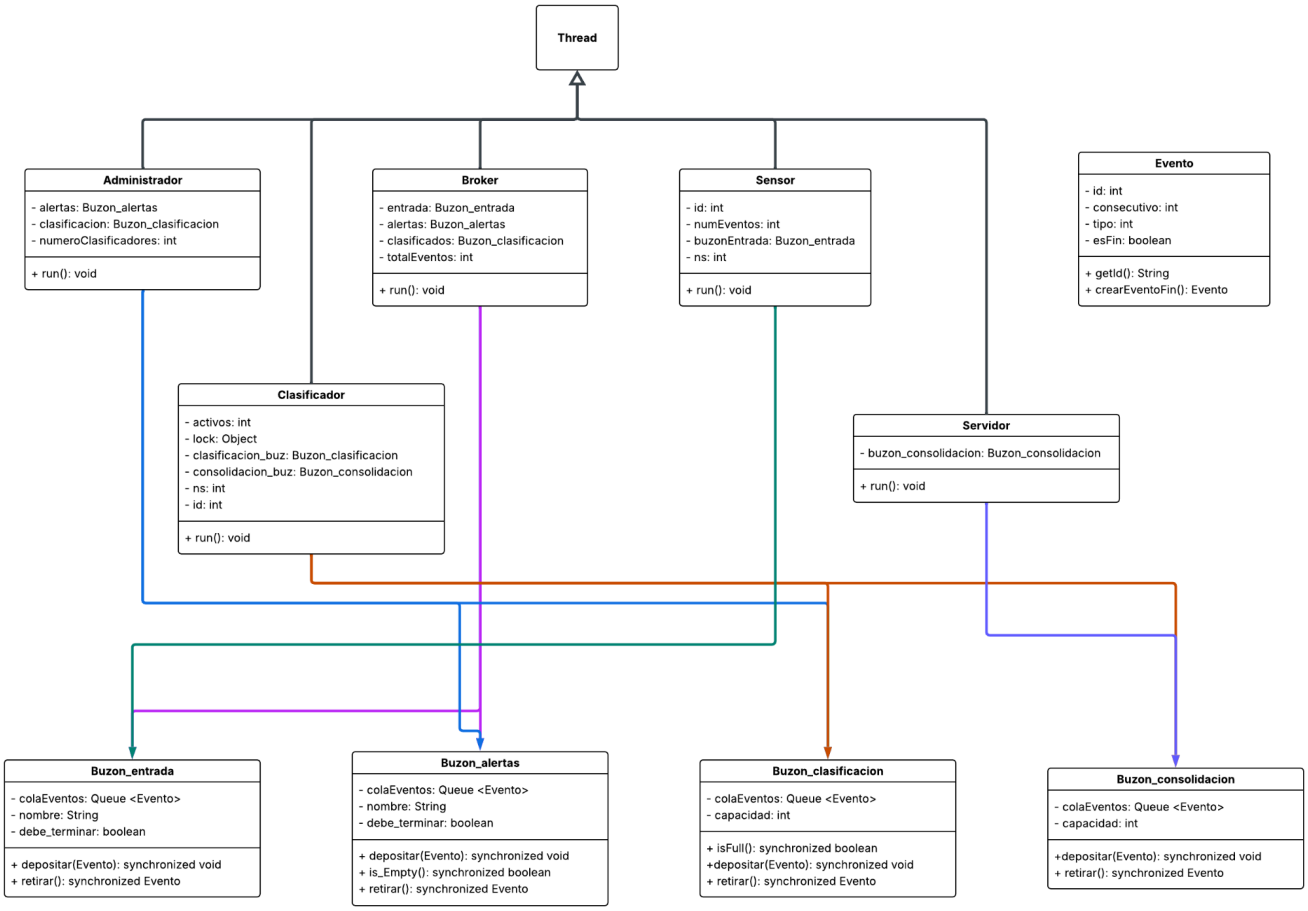

Imagen 1. Diagrama de clases del programa. Todas las clases contienen constructores que no se incluyeron como métodos en el diagrama para mayor simplicidad y entendimiento.

En el diseño de la aplicación decidimos hacer una clase por cada rol mencionado en el enunciado y cuatro buzones con diferentes funciones especificadas en las instrucciones, así conectar los threads con sus monitores sería mucho más literal y visible a la hora de probar y seguir su funcionamiento. Además, realizamos una clase Evento para que los threads que funcionan como productores puedan generar los objetos y decidimos dejarla expresada y no conectarla con todas las clases para mayor claridad a la hora de leer el diagrama e implementarlo.

Funcionamiento

Lo que los buzones hacen está descrito en la sección “sincronización”. Lo que diferencia a los ilimitados y de los limitados es que los limitados no permiten deposiciones en cuanto se alcance el límite de capacidad.

En casi todo run usamos try catch para evitar problemas con excepciones generados por funciones como yield, wait, notify, etc.

Main:

- readArchivo() lee y guarda los valores del data.txt
- Cree todos los buzones y threads y initialize los threads (pasando los ids, valores de data.txt, y los buzones que necesitan acceder)

Evento:

- Evento(): pasa todo los parámetros.
- getId(): retorna el id y el consecutivo
- crearEventoFin(): se cree un evento de fin
  Se invoque: Evento evento cuando se cree un evento “normal”
  Se invoque: Evento evento.crearEventoFin cuando se cree un evento final

Sensor:

- Sensor(): pasa el id, el buzón de entrada y todos los valores de data.txt
- run(): cree numEventos veces un evento y lo intenta depositar en el buzon_entrada

Broker:

- Broker(): pasa los parámetros al objeto y calcula el número de eventos totales (con num_eventos y num_sensores)
- run(): executa un while loop (totalEventos veces)
  Retira un evento de buzon_entrada y con Math.random decide si se debe depositar en altertas o clasificacion.
  Si pertenece a clasificación hay que usar un while loop que espera hasta que la cola en clasificación no está llena, porque clasificación es un buzón limitado.
  Incremente el número de eventos processados en cada iteración.

  Cuando se han procesado todos los eventos, intenta enviar un evento de fin al buzon_alertas.

Administrador:

- Administrador(): pasa los parámetros al objeto

- run()
  En un while(true) loop (indefinitivamente hasta que llega break){
    - Invoca yield mientras que el buzon_alertas está vacío
    - Luego(ya no vacío) retira un evento
    - Si es un evento de fin{
      Se espera con yield hasta que el buzon_clasificacion no está lleno y se deposita evento en buzon_clasificacion
      (eso se hace numeroClasificadores veces)

      Se termina el while(true) con break
      }

Si no(es un evento de fin), se define (aleatoriamente) si el evento es normal. Si es asi, se deposita en buzon_clasificacion.

Sino, no se hace nada y "evento" será sobrescrito con el siguiente evento tirado del buzón. Así se votan eventos anormales.

Clasificadores: se generan varios objetos de esa clase (El número es indicado por data.txt)

- Clasificador(): se pasan los parámetros y se cree incrementa el número de threads activos (usando un lock porque es un parámetro compartido)
- run(): En un while(true){
  Se intenta retirar un evento de clasificación

  Si no es un evento de fin{
  Se pregunta por el tipo de evento (generado aleatoriamente por los sensores). Ese número decide en cual buzón de consolidación (Clasificador tiene a un array de estos buzones) se deposita el evento.
  }

  Si es un evento de fin{
  Cada clasificador termina (y decrementa el atributo activos), el acceso solo se permite a un thread a la vez.
  El último thread para terminar (cuando activos == 0) se encarga de depositar un evento de fin a cada buzón_consolidación.

  Se termina el loop con break.
  }

  }

Servidor

- Servidor(): pasa los parámetros
- run():

  En while(true){

  Se intenta retirar un evento de buzon_consolidacion

  Si es un evento de fin:

  se termina el thread

  Sino:

  Con Thread.sleep y Math.random se simula el tiempo de lectura y procesamiento

  }

Sincronizacion

1. Sensores -> Buzón de Entrada -> Broker de eventos:

Sensores (varios)	Buzón de Entrada (illimitado)	Broker de eventos
Espera semiactiva, no tiene que esperar una condición porque el buzón es ilimitado.		El broker espera pasivamente (en el main) que llega un evento Termina cuando

	Depositar:
	Cuando se deposita el evento se despierta a los threads que estaban esperando (El broker en caso de que estaba vacío antes)
	
	Retirar:
	Espera mientras que la cola está vacía con wait
		For loop hasta que todos los eventos se procesan{
		Llama la función retirar
		}

2. Broker, Buzon de alertas, Admin

Broker	Buzon de alertas	Admin
Espera semiactiva, no tiene que esperar una condición porque el buzón es ilimitado.		Espera semiactiva (espera que hay algo para retirar)
	is_Empty: devuelve un boolean indicando si esta vacio o no Retirar: retira y devuelve un evento de la cola (sin verificar si está vacío) Depositar: el broker invoque esa función para depositar eventos	Pregunta por is_Empty en un while loop Si está vacío -> yield

3. Broker+Admin, Buzon_Clasificacion, Clasificadores

Broker+Admin	Buzon_Clasificacion (lim)	Clasificadores (varios)
Espera semiactiva (while loop with condition in thread) Tienen que esperar hasta que la cola no está llena para depositar		Esperan con espera pasiva (wait) hasta que llegó un evento para processar
while(is_Full): Thread.yield	is_Full: devuelve un boolean indicando si está llena o no Depositar: Broker+Admin depositan eventos (sin verificar si está llena) y despiertan a los Clasificadores Retirar: Los clasificadores	

	esperan con un wait hasta que no está vacío y retiran el evento	

4. Clasificadores, Buzon_consolidacion (lim), Servidores

Clasificadores	Buzon_consolidacion (lim)	Servidores
Esperan pasivamente (con wait -> en el monitor) (que no está lleno) a sus respectivos Buzones		Esperan pasivamente (con wait -> en el monitor) hasta que la cola no esté vacía.
	Retirar: espera con wait hasta que la cola no esté vacía. Despierta a los threads después de retirar. Depositar: espera con wait hasta que la cola no esté llena. Despierta a los threads después de depositar.	

Validación

Las pruebas que realizamos están basadas en las pruebas que hemos estado haciendo durante este módulo en los laboratorios. Primero, para tener claridad, creamos un data.txt e imprimimos los datos de entrada. Después, para poder seguir el proceso paso a paso y revisar las esperas, las llamadas, y quien estaba trabajando en el momento, decidimos crear logs en cada thread y así en cada acción que ejecutarán, sabríamos si hay errores o si todo corre debidamente. Al principio incluso nos dimos cuenta de que no se estaban procesando todos los eventos y que algunos no llegaban a los buzones porque solo nos devolvia que el broker procesaba 2 eventos cuando realmente estábamos esperando que procesara 6. Ahí nos dimos cuenta que al inicializar los sensores los creamos desde 0 y no desde 1 (como lo pedía el enunciado) y además no estaba cogiendo cuando era igual al número de eventos entonces estaba leyendo menos de los esperados. Después de esto empezamos a meter distintos valores para ver si funcionaba correctamente el programa.

Adicionalmente, validamos que cada sensor produjera la cantidad correcta de eventos según su identificador, que el Broker procesa todos los eventos generados, los eventos fueran correctamente enviados a los buzones correspondientes y por último que la terminación fuera la correcta. Esta última verificamos que siguiera el siguiente proceso: El Broker envía un evento de fin al Administrador, después el Administrador envía eventos de fin a los Clasificadores y el último Clasificador envía eventos de fin a los Servidores y así todos los hilos terminan únicamente al recibir su evento de fin.

Asimismo observamos no solo que los servidores procesaran correctamente los eventos sino que pudimos observar la sincronización porque podíamos observar que los hilos esperan adecuadamente cuando los buzones están vacíos o llenos.